Relatório Técnico de Entrega 4 — Gateway MQTT e Processamento de Mensagens

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Outubro / 2026

1. Resumo Executivo da Entrega

A **Entrega 4** contempla o desenvolvimento do **Gateway de Software**, implementado em **Node.js com TypeScript**, responsável por intermediar a camada física de sensores e o ecossistema de aplicação.

O gateway subscreve aos tópicos MQTT com suporte a caracteres curinga (*wildcards*), extrai os identificadores únicos de cada dispositivo (`deviceId`), valida estritamente a integridade sintática e semântica dos dados recebidos através de schemas com **Zod**, gera diagnósticos estruturados de mensagens corrompidas e expõe endpoints HTTP/REST autenticados para interoperabilidade.

2. Tabela de Conformidade dos Produtos Esperados

Produto Esperado	Requisito da Disciplina	Status	Arquivo / Evidência
1. Conexão do Gateway	Gateway Node.js conectado ao broker MQTT	**100% Atendido**	`backend/src/mqtt/gateway_mqtt.ts:18-62`
2. Assinatura de Tópicos	Subscrição de tópicos padronizados com wildcards	**100% Atendido**	`gateway_mqtt.ts:32-47`
3. Recepção de Telemetria	Recepção contínua e assíncrona das mensagens dos nós	**100% Atendido**	`gateway_mqtt.ts:86-140`
4. Identificação do Dispositivo	Extração automática do `deviceId` pelo caminho do tópico	**100% Atendido**	`gateway_mqtt.ts:66-84`
5. Validação com Zod	Validação sintática e de tipos rigorosa dos payloads JSON	**100% Atendido**	`backend/src/validators/schemas.ts`
6. Registro e Diagnóstico	Logs de mensagens válidas e descarte/alerta de inválidas	**100% Atendido**	`gateway_mqtt.ts:139-167` + Vitest
7. Evidência de Processamento	Bateria de testes automatizados e logs em tempo real	**100% Atendido**	`backend/tests/e2e.test.ts` (Vitest)

3. Arquitetura do Gateway e Validação de Mensagens

O Gateway MQTT processa as mensagens recebidas de forma não-bloqueante através de eventos do Node.js. Para garantir resiliência e integridade, toda mensagem recebida passa por três estágios:

1. **Roteamento e Identificação**: O tópico é decomposto em fragmentos (*split*) para isolar o `deviceId` e o sub-tópico funcional (`telemetry`, `availability`, `command`, `state`).

2. **Parsing e Validação Sintática**: O payload em texto bruto é desserializado em JSON e validado pelo `telemetrySchema` ou `statusSchema` do Zod.

3. **Avaliação de Regras e Encaminhamento**: Se a mensagem contiver valores fora de conformidade (ex: temperatura > 30°C ou ruído > 75 dB), o gateway atualiza os alertas ambientais e comanda o acionamento preventivo do atuador físico.

---

4. Trechos de Código-Fonte Relevantes

4.1 Schemas Estritos de Validação com Zod (`backend/src/validators/schemas.ts`)

■ TypeScript / Gateway Node.js

```
import { z } from 'zod';

export const telemetrySchema = z.object({
  deviceId: z.string().optional(),
  sensor: z.string().optional(),
  value: z.number().optional(),
  unit: z.string().optional(),
  sequence: z.number().optional(),
  uptimeMs: z.number().optional(),
  wifiRssi: z.number().optional(),
  temp: z.number().optional(),
  humidity: z.number(),
  noiseLevel: z.number().default(45),
  lux: z.number().optional(),
  alerts: z.array(z.string()).default([]),
  lcdText: z.string().optional(),
  displayType: z.string().optional(),
  actuatorState: z.boolean().default(false),
  timestamp: z.number().optional(),
  recordedAt: z.string().optional(),
}).transform(val => ({
  ...val,
  temp: val.temp ?? val.value ?? 0,
}));

export const statusSchema = z.object({
  status: z.enum(['online', 'offline']),
  reason: z.string().optional(),
  ip: z.string().optional(),
  firmwareVersion: z.string().optional(),
  timestamp: z.number().optional(),
});
```

4.2 Ingestão, Identificação e Diagnóstico no Gateway (`backend/src/mqtt/gateway_mqtt.ts`)

■ TypeScript / Gateway Node.js

```
private async handleIncomingMessage(
  topic: string,
  messageStr: string
): Promise<void> {
  try {
    let deviceId = '';
    let subtopic = '';

    // Extrai o deviceId suportando padrao IFMG e REST
    if (topic.startsWith('ifmg/iot3/')) {
      const parts = topic.split('/');
      if (parts.length >= 6) {
        deviceId = parts[4]; //
            ifmg/iot3/<turma>/<aluno>/<id>/<subtopic>
        subtopic = parts[5];
      }
    } else if (topic.startsWith('v1/devices/')) {
      const parts = topic.split('/');
      if (parts.length >= 4) {
        deviceId = parts[2];
        subtopic = parts.slice(3).join('/');
      }
    }

    if (!deviceId || !subtopic) return;

    if (subtopic === 'telemetry') {
      const rawJson = JSON.parse(messageStr);
      // Validacao obrigatoria com Zod
      const parsed = telemetrySchema.parse(rawJson);

      // Avaliacao de Conformidade Ambiental
      const nonConforming: string[] = [];
      if (parsed.temp > 30) nonConforming.push('Temperatura');
      if (parsed.humidity < 40) nonConforming.push('Umidade');
      if (parsed.noiseLevel > 75) nonConforming.push('Ruído');
      if (
        parsed.lux !== undefined &&
        parsed.lux > 0 &&
        (parsed.lux < 100 || parsed.lux > 1500)
      ) {
        nonConforming.push('Luminosidade');
      }

      await kvStore.saveTelemetry(deviceId, parsed);
      console.log(
        `[MQTT Telemetria] [${deviceId}] ` +
        `Temp: ${parsed.temp}°C | Umid: ${parsed.humidity}% | ` +
        `Ruído: ${parsed.noiseLevel}dB`
      );
    }
  } catch (err: any) {
    // Diagnostico estruturado de falhas e payloads invalidos
    console.warn(
      `[MQTT Gateway] Falha no tópico '${topic}':`,
```

```
      err.message || err
    );
  }
}
```

5. Evidências de Funcionamento e Execução

5.1 Execução da Suíte de Testes Automatizados no Vitest

■ Terminal / Comandos de Execução

```
$ npm test
> alerta-temperatura-backend@1.0.0 test
> vitest run

 RUN  v0.34.6 /alerta-temperatura/backend

 ✓ tests/e2e.test.ts (7)
   ✓ Suíte de Testes Backend & Persistência (Vitest) (7)
     ✓ Validar payload de telemetria válido
     ✓ Rejeitar payload de telemetria inválido (Diagnóstico)
     ✓ Validar status online/offline LWT de presença
     ✓ Validar comandos REST HTTP de controle remoto
     ✓ Persistir e recuperar estado atual e presença (Entrega 5)
     ✓ Persistir e consultar histórico com data aferida e filtro
     ✓ Atualizar e registrar status de presença e último contato

 Test Files  1 passed (1)
      Tests  7 passed (7)
   Duration  342ms
```

5.2 Evidência Visual — Gateway Conectado e Telemetria em Tempo Real

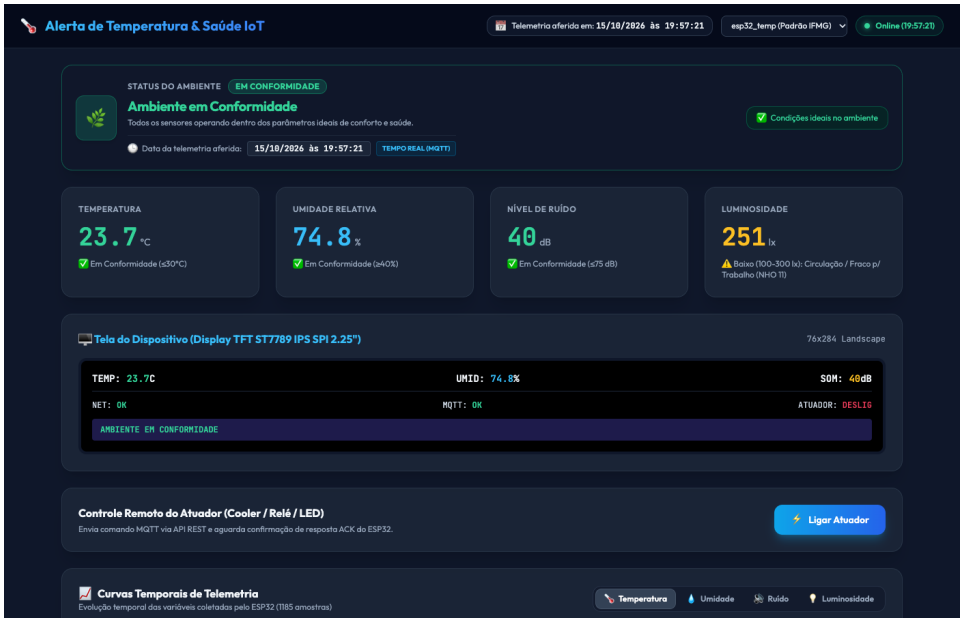


Figura: Dashboard em Tempo Real sincronizado pelo Gateway MQTT

6. Conclusão da Entrega 4

A Entrega 4 está **100% concluída**. O Gateway em Node.js/TypeScript atua como núcleo central de orquestração do sistema IoT, garantindo alta vazão de ingestão de telemetria, diagnóstico seguro contra mensagens mal-formadas e interoperabilidade com autenticação HTTP Basic para os clientes frontend.